1.- B. El operador de argumentos variables (...): En Java, para declarar un parámetro que acepte un número variable de argumentos (conocido como varargs), se utiliza una elipsis de tres puntos suspensivos (...) entre el tipo de dato y el nombre del parámetro.
2.- D. **`this()`**: En Java, se utiliza `this()` (o `this(argumentos)` si se quiere invocar un constructor sobrecargado que reciba parámetros) para invocar a otro constructor dentro de la **misma clase**.

- *Regla clave:* Si se utiliza, **debe ser obligatoriamente la primera instrucción** en el cuerpo del constructor.

3.- G. Clase X (class X {}): No define ningún constructor en su código. Por lo tanto, el compilador le asigna automáticamente un constructor por defecto.
4.- C. Error de compilación: Si intentas llamar al método pasando solo dos argumentos, el compilador no encontrará ninguna firma que coincida y detendrá el proceso de compilación generando un error (por ejemplo, indicando que el método no puede aplicarse a los argumentos proporcionados).
5.- D. Definición de inmutabilidad: Un objeto inmutable (immutable object) es aquel cuyo estado interno no puede modificarse de ninguna manera una vez que ha sido instanciado y construido. Ejemplos clásicos en Java son la clase String y las clases contenedoras (wrappers) como Integer, Double o LocalDate.
6.- A. Invocación del método print(): El operador de punto encadena la llamada .print() sobre el objeto recién creado. Este método ejecuta System.out.println(str);, imprimiendo el valor actual de la variable, que es hello.
7.- B. Impresión en main: Una vez que el método m termina su ejecución, su ámbito desaparece. Al regresar al método main, la variable i original nunca fue modificada y mantiene su valor intacto de 12, que es lo que finalmente se imprime en la consola.
8.- C. Impresión final: Al regresar al método main, la variable local a jamás fue modificada y mantiene su valor original de 20, el cual es impreso en la consola.
9.- D. **Primera llamada** (`OverloadedStaticMethods.display();`): No recibe argumentos, por lo que invoca al método `display()` sin parámetros. Esto imprime `Display with no arguments`.
Segunda llamada (`OverloadedStaticMethods.display(5);`): Recibe un valor entero (`5`), por lo que coincide con el método `display(int num)`. Esto imprime `Display with integer: 5`.
Tercera llamada (`OverloadedStaticMethods.display("Hello");`): Recibe una cadena de texto (`"Hello"`), haciendo coincidir la llamada con el método `display(String str)`. Esto imprime `Display with string: Hello`.
10.- E. **Por qué la opción e (`price.price = 4;`) soluciona el problema**:

- El primer `price` (a la izquierda del punto) accede a la variable local que contiene la referencia al objeto `Simple`.
- El segundo `.price` accede específicamente al atributo de instancia de ese objeto (el campo `float`).
- Asignar `4` (un entero) a un campo de tipo `float` es completamente válido en Java gracias a la conversión de tipo implícita (*widening primitive conversion*). Esto permite que el código compile y ejecute de manera exitosa.

11.- **Impresión final**:

- Como `bob` y `jian` comparten el mismo objeto, al evaluar `bob.name` se obtiene el valor actualizado (`"Jian"`). Por lo tanto, el programa imprime en la consola **`Bob's Name: Jian`**.

12.- B. La regla de los varargs en Java: De acuerdo con la especificación del lenguaje Java, el parámetro de argumentos variables (...) debe ser obligatoriamente el último parámetro en la declaración de la firma del método. No se permite declarar ningún otro parámetro después de un varargs.
13.- B. Impresión final: Al regresar al método stringTest() y ejecutar System.out.println(msg);, la consola imprime el valor del objeto original: Hello.
14.- A y D.

Todo constructor en Java (incluido el constructor por defecto generado por el compilador) incluye implícitamente en su primera línea una llamada al constructor sin argumentos de la clase padre mediante super();. Si la superclase no cuenta con un constructor sin argumentos, esto generaría un error de compilación.

15.- En la programación orientada a objetos, la relación "has-a" (tiene un) describe la composición o agregación, es decir, cuando una clase contiene como atributos o miembros variables a otras clases o tipos de datos.
16.- E. Este ejercicio evalúa el concepto de ámbito de las variables (variable scope), el sombreo de variables (variable shadowing) y la distinción entre variables locales y variables de instancia.
17.- A. Falta el operador de asignación (=) entre la declaración de la variable y la creación del objeto. La sintaxis correcta exige ScopeTest myScope = new ScopeTest();. La variable myScope es una variable local declarada dentro del método main. Por lo tanto, su ámbito está estrictamente limitado a main y no existe ni es visible dentro de los métodos de instancia como doStuff(), lo que genera un error de compilación por variable no encontrada (cannot find symbol).
18.-Este ejercicio evalúa la encadenación de constructores (this()), la sobrecarga de constructores y la sobrecarga de métodos (display).
19.- Este ejercicio combina conceptos de encadenamiento de constructores (this()) y sobrecarga de métodos (calculate).
20.- C. Este ejercicio evalúa cómo funciona la **encadenación de constructores** mediante `this(...)`. Recuerda que en Java, cuando un constructor llama a otro usando `this()`, esa llamada **debe ser obligatoriamente la primera instrucción** del constructor, lo que provoca que el constructor llamado se ejecute por completo antes de que el constructor original termine de ejecutar su propio cuerpo.
21.- A. Invocación recursiva de constructores: Si el compilador detecta una llamada circular o recursiva directa o indirecta entre los constructores de una clase, detiene el proceso de inmediato y genera un error de compilación por "recursive constructor invocation". Por esta razón, el código no compila y nunca llega a ejecutarse.
22.- A. El constructor sin argumentos invoca a `this(10);`, lo que redirige la ejecución al constructor con un parámetro entero.
A su vez, dentro del constructor con parámetro entero, la primera instrucción ejecuta `this();`, llamando de vuelta al constructor sin argumentos.

El compilador de Java realiza un análisis estático del flujo de inicialización de los constructores y detecta este ciclo o llamada recursiva mutua. Esto genera un error de compilación estricto (*"recursive constructor invocation"*), por lo que el código nunca llega a ejecutarse.
23.- F. Al tener éxito en la primera fase con la conversión de float a double, el compilador selecciona inmediatamente el método doIt(double d), imprimiendo en la consola double.
24.- A. public static void main(String...a) { $\rightarrow$ Sí compila. A partir de Java 5, la sintaxis de argumentos variables (varargs con ...) es totalmente válida en el método main. El compilador la traduce internamente como un arreglo de tipo String[].
public static void main(String... a) { $\rightarrow$ Sí compila. Es exactamente el mismo caso válido que el primer fragmento, solo que incluye un espacio antes del identificador.